DAY 2 · MODULE 1 · 35 MIN

Foundry IQ Deep Dive

Knowledge and grounding for your agents

Where we are in the stack

- Model — your Foundry-deployed model
- Runtime — Prompt agent, Hosted agent, or your own code + Responses API
- Actions — Modules 4–7 today
- Knowledge ← this module and Module 2
- Ops — Day 5

Knowledge answers: how does my agent know things beyond the model's training data?

What Foundry IQ is

- The managed knowledge and grounding layer of Foundry
- Turns scattered enterprise content into permission-aware, reusable knowledge bases
- Built on Azure AI Search under the hood — but you never write the retrieval code
- Same knowledge base can be shared by many agents

Source: learn.microsoft.com/azure/foundry/agents/concepts/what-is-foundry-iq

Three building blocks

Component	What it is
Knowledge source	Connection to one data store — blob container, SharePoint site, existing AI Search index, Fabric data agent, the web…
Knowledge base	Top-level resource an agent connects to. Wraps one or more knowledge sources plus retrieval parameters.
Agentic retrieval	Engine that runs multi-query pipelines across all sources in a knowledge base and returns unified, ranked results.

An agent connects to one knowledge base. That knowledge base can span many knowledge sources.

D E M O

~4 min

Attach an IQ knowledge source, portal-first

Foundry portal walkthrough: create a Foundry IQ knowledge base backed by a pre-uploaded blob container, then attach it to a docs-assistant agent. Same three objects the slide just described — knowledge source, knowledge base, agent that consumes them — but click-through instead of code. Sets attendees up for the portal setup they'll do themselves in the lab.

Runbook: demos/day2/module-1-demo-1-iq-attach-portal.md

Supported knowledge sources

Indexed

- Search index (wrap an existing one)
- Azure Blob
- OneLake
- Azure SQL (preview)
- Indexed SharePoint (preview)
- File upload (preview)

Remote

- Web (Bing) — public
- Remote SharePoint (preview)
- Fabric Data Agent (preview)
- Fabric Ontology (preview)
- Work IQ (preview)
- MCP server (preview)

Both flow through the same ranking pipeline. You mix indexed and remote in one knowledge base.

Indexed vs. remote — trade-offs

	Indexed	Remote
Content location	Inside AI Search	Stays in source system
Query latency	Low (local)	Higher (round-trip)
Freshness	On indexer schedule	Live
Setup cost	Indexer pipeline	Just a connection
Best for	Docs, blob, structured content	Live data, collab surfaces, third-party APIs
Cost model	Storage + queries	Per query against source platform

Rule of thumb: index anything you query hundreds of times per day; go remote for the rest.

Agentic retrieval — what happens on a query

- Plan — at Low or Medium effort, an LLM decomposes the question into sub-queries and picks which sources to hit. At Minimal, this step is skipped and the raw query goes straight to every source.
- Execute — sub-queries run in parallel (keyword, vector, or hybrid per source)
- Rank — unified reranker scores results across sources
- Return — top results plus source citations

This is more than 'vector search + LLM.' It's a small pipeline you get for free.

Retrieval reasoning effort — pick the level per question

Level	LLM planning	Answer synthesis	When to pick
minimal	No — single-shot search, all sources	No — extractive only	Predictable, low latency, low cost. Often the right answer for agent-consumed IQ — your agent's own model does the reasoning
low (default)	Yes — one planning pass	Optional (5K tokens)	Balance of latency and depth. Good starting point for most labs
medium	Yes — planning plus one iterative retry	Optional (10K tokens)	Deep multi-hop questions where recall matters. Select regions only

Up to 10 knowledge sources per KB on all tiers (2026-05-01-preview).

Currently preview in the Foundry and Azure portals; parts of agentic retrieval are GA on the 2026-04-01 REST API.

Source: learn.microsoft.com/azure/search/agentic-retrieval-how-to-set-retrieval-reasoning-effort

D E M O

~5 min

Query planning in slow motion

Pivot between two portals to see the same retrieval two ways. Foundry Playground: ask a multi-hop question, open the trace — one MCP tool_call is all you see. Azure AI Search chat playground for the same KB: same question, hit the debug icon on the response, and the activity log JSON shows modelQueryPlanning, three azureBlob sub-queries (with their planner-generated search strings), agenticReasoning, and modelAnswerSynthesis. Foundry gives you the summary; AI Search gives you the mechanism.

Runbook: demos/day2/module-1-demo-2-query-planning-trace.md

Permission-aware retrieval

- ACL sync — indexed sources pull access control lists from the source (SharePoint, OneLake) into the index
- Sensitivity labels — Microsoft Purview labels flow through blob / OneLake / indexed SharePoint (preview)
- Query-time enforcement — retrieval runs under the caller's Entra identity; only content the user is authorized to see comes back
- Same knowledge base, different answers per user — no per-tenant infrastructure

Single biggest reason to prefer Foundry IQ over hand-rolling RAG on AI Search for enterprise content.

When to use Foundry IQ

Reach for IQ

- Content in a supported source (SharePoint, OneLake, blob, web, Fabric…)
- Multiple agents will share the knowledge base
- Need permission-aware answers per caller
- Want managed indexing, chunking, embeddings, refresh
- Want the agentic retrieval pipeline without writing it

Reach for custom RAG (Module 2)

- Need control IQ doesn't yet give you (custom re-rankers, unusual chunk sizes, novel embeddings)
- Your source isn't a supported connector
- Already invested in an AI Search index with heavy customization
- Latency SLA below what IQ's pipeline delivers

Working with knowledge bases in MAF

From your MAF app, the knowledge base is just another dependency — no retrieval code in your agent:

```
from agent_framework import Agent
from agent_framework.foundry import FoundryChatClient
from azure.identity import AzureCliCredential

agent = Agent(
    client=FoundryChatClient(credential=AzureCliCredential()),
    name="DocsAssistant",
    instructions="Answer with citations from the docs knowledge base.",
    # knowledge_base_id wired via Foundry project configuration
)
```

Prompt / Hosted agents: KB attached in agent config. Path C: call the knowledge-base REST API or AI Search SDK.

Portal for learning, IaC for production

Same operating norm as Day 1 — production resource creation lives in code, not the portal.

- Azure CLI — az search knowledge-source create ... (verify current command surface)
- REST — PUT https://<search>.search.windows.net/knowledgeSources/<name>?api-version=2026-04-01
- Python SDK — azure-search-documents client
- Portal — fine for exploration; use it for today's lab so we don't spend the workshop on IaC

Today's lab uses the portal for the KB setup so you can focus on retrieval quality and agent behavior. Real production paths belong in IaC — that's the norm we set on Day 1.

Adjacent IQ workloads

Foundry IQ is one of three managed IQ layers.

IQ	For	In this workshop
Foundry IQ	Enterprise knowledge — SharePoint, OneLake, blob, web	← today
Fabric IQ	Analytics — OneLake, Power BI, ontologies	Available as a remote knowledge source (Fabric Data Agent, preview)
Work IQ	M365 collaboration signals — Teams, meetings, files	Available as a remote knowledge source (preview)

A knowledge base can reference all three. Not required for the workshop.

Common traps

- Forgetting the knowledge source description — LLM uses it to pick sources at low/medium effort
- Wrong retrieval effort for the workload — medium on latency-sensitive UX; minimal on multi-hop questions
- No permission model — building on blob without ACL sync, discovering per-user filtering is required later
- Chunking mismatch — default chunking on structured docs that need semantic chunking
- Ignoring citations — you skip verifying the model actually cites the retrieval, hallucinations sneak back in

Takeaways

- Foundry IQ = managed knowledge for agents. Knowledge sources, knowledge bases, agentic retrieval — three pieces.
- Indexed vs. remote is your first design choice. Index hot content, go remote for live data.
- Permission-aware retrieval at query time is the enterprise pitch. Hard to build; hard to skip.
- Retrieval reasoning effort trades latency for quality per-query.
- Create from code, not the portal.

Next: Custom RAG on AI Search — for the cases where IQ isn't the right answer yet.